Dusk Park — Design Document

Live demo: `https://g36maid.github.io/cg-final-project/Theme-Park/`

Demo video: The demonstration video exceeds the Moodle upload size limit and will be provided as a YouTube link on the GitHub repository.

Student ID: 41173058H
Department: CSIE / ME
Year: 2026
Name: 鍾詠傑 (Chung Yung-Chieh)

Figure 1: The Dusk Park meta-game at golden hour. The player stands in the central plaza with the reflecting fountain (T6), procedurally textured buildings, warm lamp point lights (T3), the dusk-gradient skybox (T5), and long dramatic shadows cast by the directional sun (T7).

1 Game Concept

Dusk Park is a 3D theme-park meta-game. The player wanders an outdoor plaza at dusk and visits four integrated sub-games: a neon *Pinball* table, a *Rubik's Cube* solver, a *3D Tetris* well, and a *Roller Coaster* ride. Three arcade sub-games *award* tokens on completion; the Roller Coaster *consumes* them per ride. The meta-objective, shown on the HUD and information board, is: *earn tokens* → *ride the coaster* ×3 → *fireworks celebration.*

The meta-game is a first-person walkthrough that concentrates every hard graphics requirement (T1–T7). Sub-games integrate via page navigation and `localStorage`-based token persistence, keeping each renderer and gameplay intact. Dusk is the narrative metaphor: the park sits at the boundary of reality and the game universes behind each door, and the warm light makes every graphics technique read clearly.

2 Scene & Visual Design

2.1 Why Dusk?

We deliberately rejected both daytime and night. Under a daytime sky the point-light contribution is visually negligible (sun washes it out) and shadows are short and flat; at night the coaster landscape is invisible and the arcade exteriors lose their context. **Dusk** is the single time of day where every required technique is simultaneously visible and dramatic:

- **Point lights + Phong** — the low ambient lets the warm lamp lights and their ambient/diffuse/specular components all read on the dark building facades.
- **Shadows** — the low sun angle stretches building shadows across the plaza, making the shadow map obvious rather than subtle.
- **Skybox** — the orange-to-purple dusk gradient is visually richer than a flat blue day sky and is cheap to generate procedurally.
- **Reflection** — the fountain water reflects the colourful sky and neon signs, which is far more legible than reflecting a uniform blue daytime sky.

2.2 Colour Palette

The palette (Table 1) is built around a warm-cool tension: warm orange/gold for the sun and lamps, cool purple/indigo for the sky and ambient, and saturated neon pink/cyan as accents that bridge the meta-game to the arcade interiors.

Table 1: Key colours and their roles.

Role	Hex	Purpose
Sky zenith	`#0A0418`	Deep indigo, anchors the top of the cubemap
Sky horizon	`#FF7A35`	Warm sunset band where the eye is drawn
Ambient	`#1F1A2E`	Cool purple fill, matches the sky's cool side
Sun (fill)	`#FFA65E`	Low-angle directional light at intensity 0.35
Lamp (point)	`#FFD98C`	Warm yellow point lights at intensity 1.8
Neon pink	`#FF4D99`	Arcade hall accent, banners
Neon cyan	`#4DDEF0`	Signage borders, fountain sparkles
Stone ground	`#59504A`	Neutral absorptive surface; takes shadows well

2.3 Objects in the Scene

The plaza contains three landmark buildings, a central fountain, an information board, street lamps, benches, planters, and decorative banners. Each major object earns its place: the **Arcade Hall** (neon-striped facade) is the portal to the three arcade sub-games; the **Coaster Station** (metal grid facade) gates the token-gated ride; the **Tour Train** station (wooden planks) anchors the far end of the plaza. The **central fountain** is the showcase for cubemap reflection (T6) and sits at the visual centre so the player encounters it immediately. The **information board** at the entrance provides the in-game instructions (D6).

3 Technical Choices

3.1 Camera & Projection

The camera uses a **65° FOV** perspective projection with near 0.1 and far 1000. The wide FOV suits an open plaza; the large far plane is required because the skybox is a 500^3 box. In first person the camera sits at **eye height 1.7 units** so the buildings (7–9 units) tower over the player. Pressing `C` switches to third-person (6 units back, 3 up), satisfying the spec's camera-control requirement (T2) and letting the player see their own long shadows.

Mouse look uses pointer lock at **0.0022 rad/px** with pitch clamped to ±89°. WASD movement uses exponentially-damped velocity (`1 - exp(-18*dt)`) so the camera glides rather than snaps—hiding discrete keypress aliasing.

3.2 Lighting

Directional sun (fill): warm orange (`[1.0, 0.65, 0.38]`) at low intensity **0.35**, direction `[-0.4, 0.35, -0.7]`. The sun is a fill, not the hero—a bright sun would flatten the contrast between lamp halos and unlit ground. The direction casts building shadows *forward across the plaza* where the player walks, making shadow mapping (T7) visible from spawn.

Point lights: four warm lamps (`[1.0, 0.85, 0.55]`, intensity **1.8**) at plaza corners (height 5) plus a cool fountain accent (intensity 0.8) at centre. Four were chosen because the 40-unit plaza needs coverage; fewer lights and the specular term would never fire on far buildings. Attenuation $\frac{1}{1+0.05d+0.005d^2}$ limits each lamp to its neighbourhood. Point lights are not shadowed (only the sun is)—a deliberate cost trade-off.

Ambient: cool purple `[0.12, 0.10, 0.18]`, tinted toward the sky rather than grey. The eye expects dusk skylight to be blue-ish; the low intensity (0.12) keeps unlit sides reading as "in shadow."

3.3 Phong Materials

The shader computes, per fragment: `colour` $= k_a \cdot \text{base} \cdot \text{ambient} + \sum_i (k_d \cdot \text{NdotL} + k_s \cdot \text{spec}) \cdot \text{light}_i$. Each major surface has a hand-tuned material (Table 2). The reasoning for each follows.

Table 2: Phong material parameters per surface. Values are linear-RGB coefficients.

Surface	k_a	k_d	k_s	**shin.**	**Rationale**
Stone ground	0.15	0.35	0.15	8	Rough, absorptive; low diffuse so lamps must light it; very low shininess for a matte look
Buildings	0.20	0.55	0.30	32	Painted concrete; moderate specular so lamp halos glance off walls; shininess 32 gives a medium-tight highlight
Metal (lamps, station)	0.10	0.30	0.90	128	High specular (0.90) and high shininess (128) so lamps produce a tight bright glint—this is what "metal" should look like under a point light; low diffuse so the metal reads as reflective rather than painted
Fountain orb	0.10	0.30	0.90	128	Same metal preset as the lamps, tinted blue, to read as a polished sphere catching the cool fountain light
Arcade facade	0.14	0.42	0.55	48	Slightly higher specular than generic buildings so the neon texture's bright bands catch the lamps

Per-parameter justification for key surfaces:

- **Ground**: k_a=0.15 keeps it barely visible in shadow; k_d=0.35 is deliberately low so the ground *receives* light without competing with lamps; k_s=0.15 / shininess 8 gives a broad matte sheen for weathered stone.
- **Metal** (lamps, station, fountain orb): k_s=0.90 / shininess 128 produces the tight bright glint that defines polished metal under a point light. Low k_d=0.30 keeps metal dark between highlights, making them read as reflections.
- **Buildings**: k_d=0.55 carries texture colour; k_s=0.30 / shininess 32 shows a lamp halo on the wall without looking metallic.

3.4 Textures

Procedural stone ground (T4): a 256^2 canvas texture with per-pixel noise, 32-px grout lines, and crack polylines, tiled $\approx 33\times$ across the 200^2 plane (`REPEAT`, mipmaps on). Procedural avoids both asset dependencies and visible seams; the grout lines give scale ("paved plaza", not "infinite gravel").

Building facades: each building gets a distinct procedural texture that telegraphs its sub-game—neon bands for the Arcade Hall, brushed-metal grid for the Coaster Station, wooden planks for the Tour Train.

Skybox (T5): a 500^3 box with a procedural cubemap. Side faces are vertical gradients (indigo → orange sunset → dark ground); the top face has 120 stars. Uses `cullFace = GL_FRONT`, `depthWrite = false`, `frustumCulled = false`.

3.5 Shadow Mapping (T7)

OGL's `Shadow` extra renders a 2048^2 **depth map** from an orthographic sun camera (± 30 frustum, near 0.5, far 80, positioned 40 units up the sun vector). The Phong shader projects each fragment into sun-clip space and applies a shadow factor of **0.35** (shadowed regions keep 35% of the sun contribution) with bias **0.002**. We chose 0.35 over full black because dusk shadows are soft-filled by skylight; pitch-black shadows would read as high noon.

3.6 Cubemap Reflection on the Fountain (T6)

The fountain water reflects the skybox cubemap via $\mathbf{R} = \text{reflect}(\mathbf{I}, \mathbf{N})$. A Fresnel term $F = (1 - \max(-\mathbf{I} \cdot \mathbf{N}, 0))^3$ modulates the mix: grazing angles show mostly reflection (strength 0.9), looking straight down shows the water's dark teal (strength 0.45). This matches real water behaviour and makes the dusk sky paint itself onto the fountain.

3.7 Token Economy & Game Mechanics (D1–D6)

All six mechanics dimensions are implemented: **D1** clear objective (earn → ride ×3 → celebration); **D2** collision/interaction (AABB colliders on every building, lamp, bench, planter; E-key proximity triggers at each door); **D3** scoring visible (HUD shows token count and coaster rides); **D4** win/lose (soft win: 3 rides triggers a fireworks celebration overlay; no lose state, by design); **D5** feedback (token-gain/loss toasts, proximity prompts, celebration animation); **D6** instructions (entrance information board + M-key help panel listing objective, controls, and facility roles).

4 Integrated Sub-Games

The four sub-games are integrated via page navigation (`location.href`) with a `?from=hub` URL parameter. Token state persists in `localStorage['dusk-park-state']`. A shared hooks module (`Theme-Park/src/meta/hooks.js`) injects a "return to park" button and handles payout events, so sub-game physics and rendering are untouched.

Figure 2: 3D Pinball: hand-rolled 2D physics on a tilted table, a PBR metal ball with cubemap reflection, Bloom post-processing, and Web Audio SFX. Awards 10 tokens on game over.

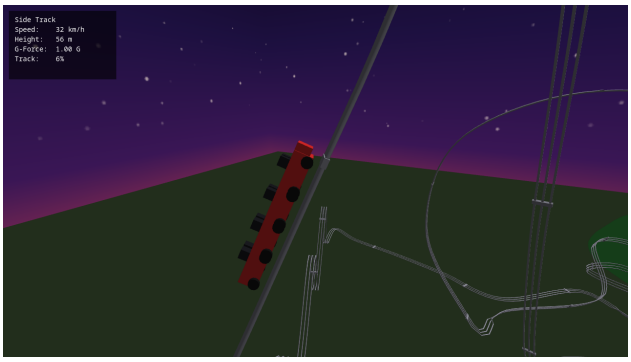

Figure 3: Roller Coaster: Catmull-Rom spline track with Frenet–Serret (TNB) frame orientation, energy-conservation physics, and a 2D-canvas HUD (speed / height / G-force / progress). Consumes 10 tokens per ride.

Figure 4: Rubik's Cube: raycast face-picking, drag-to-rotate with `easeOutBack` snap, a layer-by-layer auto-solver, and a live 2D cross-net. Awards 10 tokens on solve.

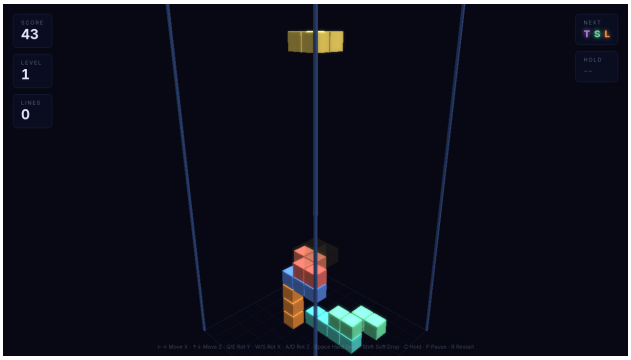

Figure 5: 3D Tetris: a $10 \times 20 \times 10$ volumetric well with SDF round-edge blocks, three-axis rotation (wall-kick corrected), ghost-piece preview, Bloom post-processing, and a grey-scale game-over fade. Awards 10 tokens on game over.

5 Challenges and Solutions

5.1 The Skybox Was Invisible: the `mat3 + xyww` Trap

Symptom. The 500^3 skybox was completely invisible; even a pure-red fragment shader produced no pixels. No WebGL errors.

Root cause. The textbook trick `gl_Position = (proj * mat3(mv) * pos).xyww` forces $z{=}w$ to place the box on the far plane. This assumes the camera is at the box centre. Our camera is at `[0, 1.7, 18]`—for vertices

behind it, $w < 0$, so the clip test $-w \le z \le w$ fails and half the triangles are clipped away.

Fix. Standard projection with a giant box centred at the origin, `cullFace = GL_FRONT`, `depthWrite = false`. Robust for any camera position.

5.2 OGL Array-Uniform Naming

Symptom. `uniform vec3 uLightPos[8]` logged "not supplied" despite passing values.

Root cause. OGL parses `uLightPos[0]` and looks up the base name `uLightPos`; keying as `'uLightPos[0]'` misses. Also, `Float32Array` fails `Array.isArray()`, so OGL misinterprets it as a struct.

Fix. Key by base name; pass plain `Array` (not `Float32Array`).

5.3 ES-Module Import Paths

Symptom. `../../ogl/src/index.js` 404'd when serving from `Theme-Park/` alone.

Fix. Serve from repo root so all sibling directories resolve; add a `Theme-Park/ogl -> ../ogl` symlink for standalone serving.

6 Most Proud Of

What we are most proud of is the **project's trajectory**. The four sub-games did not start as components of a theme park—they began as independent experiments while learning OGL: a Pinball table for hand-rolled physics and PBR, a Rubik's Cube for quaternion layer-rotation and raycast picking, a 3D Tetris for SDF shaders and Bloom, and a Roller Coaster for Catmull-Rom splines and Frenet–Serret framing. Each explored a different corner of real-time graphics.

The insight that turned four demos into one game was the **meta-game**: instead of rewriting them into a single renderer, we made the park itself the integration layer. A first-person plaza where each door leads to a different game universe, connected by a shared token economy. The four games kept their physics, rendering, and gameplay untouched; the meta-game glued them together with page navigation, `localStorage`, and a shared hooks module. What was originally "four ways to explore a library" became *Dusk Park*—one cohesive game set against a dusk sky that makes every graphics technique visible in a single frame.